HTML-CSS-06 第6回_まとめ

第6回の目標

- **今までの内容を復習**する（過去にやったことをかなり使う）
- **画像のトリミング**の仕方を知る
- **マーカーのような下線**をCSSで引けるようになる

画像を指定サイズでトリミングする（object-fit）

画像の比率を変えずに、指定した幅・高さの枠にきれいに収めたいときに使う。

```
.sample_img {
  height: 350px;
  object-fit: cover;
}
```

- `object-fit: cover;` をつけると、指定した高さ・幅の枠に画像の比率を保ったままトリミングして収められる（はみ出す部分は見えなくなる）
- `img` タグに直接高さを指定しただけでは画像が縦横比を保たず歪んでしまうため、`object-fit` と組み合わせるのが定番
- `object-fit` は `img` タグに書く

値	動き
`cover`	縦横比を保ったまま、枠を完全に覆うようにリサイズ（はみ出た部分は切れる）
`contain`	縦横比を保ったまま、枠に収まるようにリサイズ（余白ができることがある）

- CSSだけで画像をトリミングできるobject-fit

calc()で要素のサイズを計算する

```
.sample_line {
  width: calc(100% - 140px);
}
```

- `calc()`関数を使うと要素のサイズを計算式で指定できる → 画面幅などが変わっても柔軟にサイズが変化する
- 📎 calc()を使って値を計算しよう

疑似要素でラベル装飾をつける

見出し上の小さいラベルのような装飾は、`::before`疑似要素で`content`を出し分けて作ることができる。

```
.sample-title {
  position: relative;
}

.sample-title::before {
  display: block;
  color: var(--accent-color);
  font-size: 16px;
  font-weight: normal;
  letter-spacing: 0.3em;
}

.sample-title__company::before {
  content: "COMPANY";
}

.sample-title__blog::before {
  content: "BLOG";
}
```

- 同じベースクラス（`.sample-title`）に、モディファイアクラス（`__company`／`__blog`）を組み合わせて`content`の中身だけを切り替えている（第2回の複数クラスの考え方と同じ）

マーカーのような下線をlinear-gradientで作る

`linear-gradient()`の色の切り替わり位置を調整すると、テキストの下側だけに色がついた「マーカーで引いたような下線」を再現できる。

```
.sample_emphasis {
  background: linear-gradient(
```

```
    transparent 0%,
    transparent 75%,
    var(--accent-color) 75%,
    var(--accent-color) 100%
  );
}
```

- 0%〜75%までを透明（`transparent`）、75%〜100%だけを色付きにすることで、テキストの下25%だけに色の帯が入り、マーカーのように見える
- パーセントの数値を変えると、下線の太さ（どこから色がつくか）を調整できる
- 上から見て「△%〜□%が何色」という考え方で指定する
- `0%`と`100%`は省略できる（`linear-gradient(transparent 70%, #43cfe8 70%)`でも同じ）
- カンマを忘れるとそれだけで表示されないことがあるので注意

CSSで図形を作る

正円

```
.circle_item {
  width: 200px;
  height: 200px;
  border-radius: 50%; /* 正方形に50%で正円になる */
  background-color: #e84343;
}
```

- `border-radius: 100vh;`は、要素のサイズいっぱいに角を丸める指定でよく使う（カプセル型）
- border-radiusで円・角丸を作る／border-radiusで様々な角丸に挑戦

アイコンを丸枠で囲む

1. 幅と高さを指定する（例：`width: 96px; height: 96px;`）
2. 枠線で囲う（`border: 4px solid var(--primary-color);`）
3. 角を丸くする（`border-radius: 50%;`）
4. アイコンを枠の上下左右中央に置く（`display: flex; align-items: center; justify-content: center;`）

三角形

```
.triangle_item {
  border-width: 100px;
  border-style: solid;
  border-color: transparent transparent #436ae8 transparent; /* 下の辺だけ色をつける → 上向きの三角形 */
}
```

- 原理が難しければ 三角形ジェネレーターを使う。図形のジェネレーターは他にもたくさんある
- CSSでいろいろな図形を作ろう／おもしろいCSS検索BOX（＋α）

横スクロールを実装する

ボタンなどの一覧が画面幅に収まらず、横にスクロールさせたいときのパターン。

```
.sample_wrapper {
  width: 100%;
  overflow-x: scroll;
}

.sample_list {
  width: calc(343px * 4 + 32px * 3);
  display: flex;
  flex-wrap: wrap;
  gap: 16px 32px;
}
```

- 親要素に`overflow-x: scroll;`をかけ、中の`ul`などの幅を`calc()`で「アイテム幅×個数＋余白×個数」分だけ明示的に広くとることで、横スクロールが実現する
- `flex-wrap: wrap;`をかけると、`flex`の子要素が幅に収まらないときに自動で折り返される（第1回の表示形式参照）
- 【CSS】overflowの使い方解説！要素のはみ出し解決

疑似要素でバツ印（開閉アイコン）を作る

クリックして開閉するアコーディオンなどのアイコンは、`::before`で縦棒・`::after`で横棒を同じ位置に重ねて作ることができる。

```
.sample_mark {
  width: 20px;
```

```
  height: 20px;
  position: relative;
  flex-shrink: 0;
}

.sample_mark::before {
  content: "";
  position: absolute;
  top: 50%;
  left: 50%;
  transform: translate(-50%, -50%);
  width: 2px;
  height: 100%;
  background: var(--black-color);
}

.sample_mark::after {
  content: "";
  position: absolute;
  top: 50%;
  left: 50%;
  transform: translate(-50%, -50%);
  width: 100%;
  height: 2px;
  background: var(--black-color);
}
```

- 開閉マーク（バツ印）は、`::before`で縦棒・`::after`で横棒を同じ位置（`top: 50%; left: 50%;` + `translate(-50%, -50%)`）に重ねて作る（第5回の上下左右中央寄せと同じ考え方）
- `flex-shrink: 0;`をつけると、`flex`の他の要素が広がってもこのアイコンの大きさは縮まない（第3回参照）
- `:not(:first-child)`のように「最初の要素以外」を指定できる疑似クラスもある（2個目以降にだけ余白をつけたい時などに便利、第1回参照）
- `::before`と`::after`の使い分け
  - `position`をつけない場合：中身の前に出るか後に出るかが変わるので、使い分けが必要
  - `position: absolute;`をつける場合：要素が浮いて`relative`を基準に配置されるので、どちらでもよい
- 片方の軸だけ中央寄せする書き方（スニペット登録しておくと便利）

```
/* 左右だけ中央寄せ（X軸） */
position: absolute;
left: 50%;
transform: translateX(-50%);

/* 上下だけ中央寄せ（Y軸） */
position: absolute;
top: 50%;
transform: translateY(-50%);
```

※ KV（ファーストビュー）への色付きフィルター・上下左右中央寄せのテクニックは第5回で扱った内容と同じなので、そちらを参照。

共通部分はコピーで効率アップ

- ヘッダー・CTA・フッターはトップページと同じデザインなので、コピー＆ペーストで時間を短縮する
- KVのようにほとんど同じデザインのものも、コピーして必要な部分だけ直す

下層ページのKVをモジュールにする（m_kv）

サービスページなどの下層ページのKVは、他のページでも使い回すことを想定して`m_`のモジュールとして作る。似たスタイルの箇所（トップページのKV）からコードを真似できないか、まず探すのがポイント。

```
.m_kv {
  height: 320px;
  background: linear-gradient(rgba(51, 51, 51, 0.2), rgba(51, 51, 51, 0.2)),
    url(../img/bg_kv.jpg) center 22% / cover;
  position: relative;
}

.m_kv-box {
  position: absolute;
  top: 50%;
  left: 50%;
  transform: translate(-50%, -50%);
  text-align: center;
```

```
  width: 100%;
}
```

- `background-position` の `center 22%` は「横は中央、縦は上から22%の位置」を基準に画像を表示する指定。キーワード（`top` など）だけでなく、%で細かく位置を調整できる
- グラデーションの書き方（サルワカ）
- ICS MEDIAの参考記事（授業ページで紹介）

Q&A（開いた状態）を作る

- 1つだけ作成して、残りは複製すればよい（クリックで開閉する動きは、のちにJavaScriptで実装する想定）
- 質問部分は `button` 要素で作り、`display: flex; justify-content: space-between;` で「Q・質問文」と「開閉マーク」を両端に並べる
- `Q` ／ `A` のアイコンは同じベースクラスにModifier（`__q` ／ `__a`）で色だけを変える
- 開閉マーク（＋／×）は前述の「疑似要素でバツ印（開閉アイコン）を作る」で作る

本日のまとめ

1. 疑似要素の使い方を思い出せた
2. `object-fit: cover;` で画像のトリミングができる
3. マーカーのような下線は `linear-gradient` で作れる
4. `overflow` プロパティははみ出した要素に影響を与えられる（`overflow-x: scroll;` で横スクロール）
5. `flex` の子要素の折り返しは `flex-wrap` を使う

参考記事

- 【CSS】flexアイテムの折り返し！flex-wrapの使い方を解説
- linear-gradient()を使ってグラデーションしてみよう
- calc()を使って値を計算しよう
- CSSでいろいろな図形を作ろう
- 【CSS】overflowの使い方解説！要素のはみ出し解決
- CSSの擬似要素で擬似的に要素を追加しよう
- positionとtransformで要素を上下左右中央寄せしよう
- 複数背景で画像の上に半透明のレイヤーを載せる方法
- グラデーションの書き方（サルワカ）
- ICS MEDIAの参考記事（授業ページで紹介）
- 【復習】横スクロールを実装する（よく使うCSSプロパティ）